Plexus

A blueprint for differentiable tissue

Janelia Research Campus

August 6, 2026

Abstract

We introduce *Plexus*, a typed intermediate representation for biological mechanisms. Rather than defining yet another simulator, Plexus defines a common language whose primary objects are biological operators. Every operator specifies the biological entities it acts upon, the states it requires, the relations between these entities, and one or more numerical implementations. The Plexus research program has three objectives. First, to define an algebra of biological operators. Second, to decompose the existing corpus of simulation models into this common language. Third, to construct differentiable implementations of these operators so that mechanistic models can be directly inferred from biological data.

1 Introduction

Computational biology has produced a rich ecosystem of simulation frameworks. Neural simulators model electrical activity, reaction–diffusion systems model chemical transport, particle and continuum methods model mechanics, while vertex, Cellular Potts and active-matter models describe multicellular tissues. These frameworks remain however largely incompatible. Combining electrical signalling, mechanics, metabolism and development often requires coupling multiple specialized simulators whose underlying abstractions differ fundamentally. Each simulator introduces its own representation of biological mechanisms, making models difficult to compare, compose and extend. This fragmentation is not primarily a numerical problem but more a language problem.

Instead of treating simulation frameworks as fundamental, we treat biological mechanisms as the primary objects. To this aim, Plexus introduces an intermediate representation whose central elements are biological operators. Operators describe mechanisms such as adhesion, diffusion, division, chemotaxis, electrical propagation or active stress independently of their numerical implementation. In this perspective, biological modeling becomes analogous to compiler design. Just as modern compilers separate programming languages from their implementations through an intermediate representation, Plexus separates biological mechanisms from their numerical implementations.

This viewpoint defines a broader research program. First, Plexus seeks to construct an algebra of biological operators whose elements can be composed into complex dynamical systems, compared through their observable consequences, and expanded as new mechanisms are discovered. Second, Plexus aims to extract this operator algebra from the existing corpus of biological simulation models. Rather than viewing current frameworks as competing simulators, we regard them as repositories of reusable mechanisms. By decomposing these models into a common language, we seek to identify a compact vocabulary capable of expressing a wide range of biological dynamics across scales. Third, Plexus constructs differentiable counterparts of these operator compositions.

Differentiability allows mechanistic models to be fitted directly to biological data, residuals to be interpreted mechanistically, and missing operators intuited.

Finally, because biological mechanisms are represented in a formal language rather than embedded in simulator-specific code, they become amenable to automated reasoning. The explicit separation between biological semantics and numerical implementations allows agentic systems to operate directly on the language itself, composing, comparing and optimizing mechanistic models without modifying simulator code. The search for biological explanations therefore becomes a search over model structures rather than simulator implementations.

2 The language

Plexus defines a typed language for expressing biological mechanisms independently of their numerical realization. The language is built around four fundamental concepts: *operators*, *sets*, *states*, and *fields*. Biological models are therefore represented as compositions of operators acting on discrete and continuous biological objects rather than simulator-specific algorithms.

2.1 Operators

Operators are the fundamental objects of the language. An operator represents a single biological mechanism, such as adhesion, chemotaxis, diffusion, electrical propagation, active stress, metabolism, growth, or cell division. Every operator is defined by a typed signature and one or more numerical implementations. The signature specifies *what* biological transformation is performed, whereas implementations specify *how* it is numerically realized. The signature specifies

- the input sets,
- the output sets,
- the state variables consumed and produced,
- the maps relating the participating sets.

Maps define the relations required by the mechanism. They determine, for example, which particule belong to which cell, which synapses connect which neurons, or how cells relate to a continuous field. Plexus incorporates maps into the operator, making operators typed morphisms between biological sets.

2.2 Sets

Sets represent collections of biological entities. Examples include genes, proteins, particles, cells, neurons, synapses, tissues, organs and organisms. Sets naturally organize into hierarchies reflecting biological organization. Genes regulate proteins, proteins determine cellular behaviour, cells assemble into tissues, tissues form organs, and organs compose organisms. Each level is represented uniformly as a set, allowing the same language to describe intracellular signalling, multicellular mechanics, developmental processes and organ-level physiology.

2.3 States

Every element of a set carries a state describing its current biological condition. While sets define *what* biological entities exist, states define *what each entity carries*. The state schema is specific

to each set. A particle may carry position, velocity and deformation gradient; a neuron membrane voltage and calcium; a cell polarity, stiffness and gene-expression levels; a synapse conductance and plastic weight. Operators transform states. They consume selected state variables and produce updates to others, while leaving the underlying biological entities unchanged.

2.4 Fields

Fields represent continuous quantities defined over space rather than over discrete biological entities. Examples include morphogen concentrations, pressure, extracellular signalling molecules, electric potentials, and material properties. Fields complement sets. While sets describe discrete biological entities, fields describe the continuous environment in which these entities evolve. Operators couple sets and fields through explicit maps relating discrete entities to continuous spatial representations.

3 The operator algebra

The language introduced in the previous section defines the objects manipulated by Plexus. We now define the elementary operator families from which biological models are constructed. The central hypothesis of Plexus is that a large fraction of existing biological simulation models can be expressed as compositions of a small number of elementary operator families. Rather than introducing a new operator for every biological phenomenon, Plexus identifies a compact operator algebra that can be reused across mechanics, signalling, metabolism, development and physiology.

The proposed operator algebra consists of eight elementary families (Fig. 1):

- **Lateral**: interactions within a biological set.
- **Aggregate**: many-to-one transformations across biological scales.
- **Broadcast**: one-to-many transformations across biological scales.
- **Exchange**: transformations between sets and fields.
- **Rewire**: modification of relations between biological entities.
- **Divide**: creation of biological entities.
- **Die**: removal of biological entities.
- **Seed**: establishment of the initial state.

The first four families transform biological state while preserving the underlying biological organization. Rewire modifies the interaction graph, whereas Divide and Die modify the biological entities themselves. Seed writes the state a model starts from. Together they form the elementary computational vocabulary of Plexus.

Seed is separated from Divide and Die — with which it shares a mechanism, since all three write state directly rather than returning a delta — because it differs in *when*. Divide and Die act throughout a trajectory; Seed acts only at its opening. Treating that distinction as a naming convention rather than a kind has a concrete cost. In an automated model-search campaign on a growing epithelial vesicle, an initial condition for a reaction–diffusion field was re-applied on every timestep rather than once, which converts it from an initial condition into a moving boundary condition and silently overwrites the state of any operator writing to the same channel. Both spellings were legal specifications and nothing in the language could distinguish them; the resulting

runs reported a mechanism as refuted eight times across thirteen rounds when the operator under test had in fact never reached the state. Because Seed is a kind, the runtime imposes its window, and a per-timestep initial condition is not expressible.

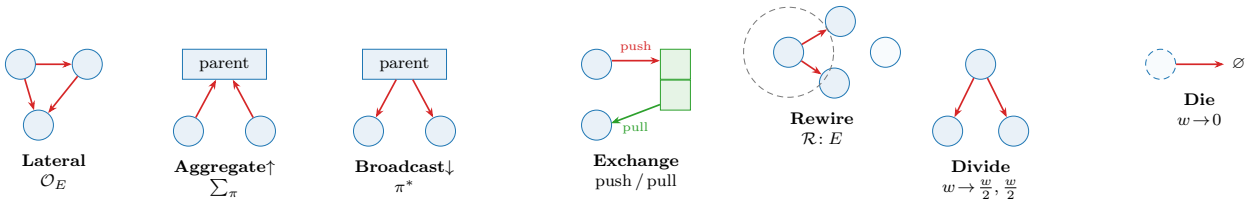

Figure 1: The elementary operator algebra of Plexus. Lateral, Aggregate, Broadcast and Exchange transform biological state. Rewire modifies biological relations, whereas Divide and Die modify biological entities. Complex biological models are obtained by composing these elementary operator families.

Lateral. Lateral operators describe interactions between entities belonging to the same set. Typical examples include mechanical forces between particles, adhesion between neighbouring cells, electrical communication between neurons, or biochemical interactions within a metabolic network.

Aggregate. Aggregate operators propagate information from many entities to a higher level of biological organization. Examples include computing cellular properties from their constituent particles or accumulating synaptic currents acting on a neuron.

Broadcast. Broadcast operators perform the complementary transformation, propagating information from higher organizational levels toward their constituents. Aggregate and Broadcast therefore define the upward and downward flow of information across biological scales.

Exchange. Exchange operators couple discrete biological entities with continuous fields. Typical examples include secretion, sensing, transport, uptake, or particle–grid transfer.

Rewire. Rewire operators modify the relations between biological entities without changing the entities themselves. They update neighbour graphs, contact networks, developmental connectivity or other interaction structures, determining which entities participate in subsequent operator evaluations.

Divide and Die. Living systems continuously modify their own composition. Divide creates new biological entities, whereas Die removes existing ones. These operators modify the biological organization itself rather than only its state.

4 Composing biological mechanisms

Biological models are expressed as compositions of elementary operators. Rather than embedding all mechanisms within a monolithic simulator, Plexus represents a model as an ordered sequence of operator applications. This sequence, called the *schedule*, specifies both the order and the frequency with which operators are evaluated. Formally, one simulation step is the composition

$$\Phi = \mathcal{O}_n \circ \cdots \circ \mathcal{O}_1,$$

whose repeated application generates the system dynamics

$$x_{t+1} = \Phi(x_t).$$

The schedule therefore describes the temporal organization of biological mechanisms. Different operators may evolve on different time scales. Mechanical interactions may be evaluated every simulation step, whereas diffusion, growth or cell division may occur less frequently. A biological model therefore becomes a multi-rate composition of biological mechanisms rather than a monolithic numerical solver.

Operators are pure transformations. Rather than modifying biological state directly, every operator returns its contribution to the system evolution. If operator $\mathcal{O}_i$ produces the increment Δ_i, the complete update is obtained by

$$\Delta = \sum_i \Delta_i,$$

before being passed to the numerical implementation. This formulation has two important consequences. First, biological mechanisms remain independent and composable: introducing a new mechanism simply requires adding a new operator to the schedule. Second, the language remains independent of its numerical realization. The schedule specifies the biological model, whereas the numerical implementation specifies how that model is evaluated.

5 Numerical implementations

The operator algebra defines the biological semantics of a model independently of its numerical realization. Every operator may admit multiple implementations satisfying the same typed signature. These implementations differ in numerical realization while preserving identical biological semantics. Selecting an implementation is therefore a numerical decision rather than a biological one.

This abstraction naturally accommodates a broad range of computational methods. Mechanical operators may be implemented using Material Point Methods, finite elements, vertex models, phase-field methods, Cellular Potts models or optimal-transport formulations. Signalling operators may employ Hodgkin–Huxley equations, graph neural networks or Neural ODEs. Likewise, field operators may use finite differences, finite elements, neural implicit representations or spectral methods. These implementations coexist within the same language because they satisfy identical operator contracts.

Different implementations may also employ different numerical integration schemes. Depending on the operator, implementations may evolve first-order states such as concentrations or membrane voltages, second-order states such as mechanical accelerations, or rely on explicit, implicit or adaptive solvers. These choices affect computational efficiency and numerical accuracy but not the biological interpretation of the model.

This distinction fundamentally changes how biological simulations are compared. Traditional frameworks compare complete simulators, intertwining biological mechanisms with numerical algorithms. Plexus instead separates these two questions. Biological models are compared through the operators they compose, whereas numerical methods are compared through the implementations realizing those operators.

Because biological mechanisms are represented in a formal language rather than embedded in simulator-specific code, they become amenable to automated reasoning. The explicit separation between biological semantics and numerical implementations allows agentic systems to operate directly on the language itself, composing, comparing and optimizing mechanistic models without modifying simulator code. The search for biological explanations therefore becomes a search over model structures rather than simulator implementations.

6 Mechanistic inverse modelling

One of the primary motivations for introducing a formal language of biological mechanisms is to identify mechanistic models directly from experimental observations. Rather than calibrating simulations manually, the objective is to infer the parameters, spatial organization and constitutive laws governing biological systems from imaging data.

Plexus makes this possible by associating every biological operator with one or more differentiable implementations satisfying the same typed signature. Differentiability therefore becomes a property of the language rather than of a particular simulator. Regardless of the numerical backend, gradients propagate through compositions of explicit biological operators rather than through monolithic simulation code.

This distinction fundamentally changes the role of inverse modelling. Traditional approaches assume that the underlying mechanism is already correct and seek the parameter values that best reproduce the observations. Such parameter optimization is effective when the forward model is complete, but it becomes misleading when important biological mechanisms are absent. Optimizing existing parameters may improve the fit while merely compensating for missing mechanisms.

In Plexus, parameters remain attached to explicit biological operators rather than to opaque numerical models. Mechanical properties belong to mechanical operators, reaction rates to biochemical operators, conductances to signalling operators, and spatial fields to the mechanisms generating them. Parameter estimation therefore preserves the mechanistic interpretation of the model.

Residuals likewise acquire a mechanistic interpretation. A persistent mismatch between simulation and experiment no longer indicates only poor parameter values; it may also indicate that the current operator composition is unable to explain the observations. Differentiability thus provides more than efficient optimization: it enables quantitative comparison between competing mechanistic models while preserving their explicit biological structure.

Crucially, because the operator composition is differentiable end to end, the objective need not span the entire system: a loss may be *local* in both space and time. It can be evaluated on a chosen *subset of entities* — a region of interest such as the extrusion front where activated and quiescent cells meet — over a *short rollout* of a few tens of steps, rather than on every cell across the whole trajectory. Gradients still propagate through the shared operator composition, so a *local* objective fine-tunes precisely the mechanism responsible for a local behaviour — the shape of a growth front, the sharpness of a domain boundary, the timing of a rearrangement — without the averaging-out

of a global fit and, decisively, without the prohibitive cost of differentiating a long, whole-system rollout. This last point is what makes the approach practical at all: a loss spanning every cell over hundreds of frames unrolls, under reverse-mode differentiation, into a backpropagation graph whose memory and compute scale as cells × steps and quickly become intractable, whereas a loss confined to a small set of entities over a short window keeps that gradient graph small. *Locality* is therefore the enabling property of differentiable refinement on large, long-lived tissues: the mechanism is global and shared, but the *evidence* constraining any single operator is usually confined to a small, transient region, and the loss should be placed exactly there.

7 Agentic mechanistic discovery

> *"What I cannot create, I do not understand."*
> *"Know how to solve every problem that has been solved."*
>
> — Richard P. Feynman[1]

Feynman's two lines frame this section. A biological mechanism is understood only when an agent can *create* it — assemble it from operators until it reproduces the observations — and new discovery begins only once the agent can already re-derive every model that has been solved. The three loops below make this operational, moving from reconstructing what is already known toward constructing the mechanisms that existing models cannot yet express.

The explicit representation of biological mechanisms transforms scientific discovery from a parameter optimization problem into a language exploration problem. Because mechanisms are represented as explicit operators rather than embedded in simulator-specific code, they become directly accessible to agentic systems.

Plexus proposes a three-loop strategy for mechanistic discovery. Each loop operates at a different level of abstraction, progressively moving from existing models toward new biological knowledge.

Loop I: Understanding. The first loop takes an existing model — a paper and its repository — and decomposes it into Plexus operators, sets, states and fields, so the biological mechanisms it implements become explicit. That decomposition is necessary, but it is only the *entry*: it makes the model's *levers* explicit without yet explaining them. Understanding is earned afterwards, by *interrogating* the decomposition as a causal system — learning what each operator and parameter does on its own and, the harder part, what it does in combination, since mixtures rarely surrender their causal structure to inspection. The driving force is therefore not translation but a *hypothesis → validate/falsify* cycle run on the decomposed model: each step states a falsifiable claim about a lever, tests it against a dedicated metric, and accumulates the causal map that results. A useful operating point is a rough **70/30 balance of validation to falsification**: pure validation only ever confirms hypotheses one already believes (trivial, near-zero information), while pure falsification only ever probes the improbable (which mostly breaks as expected and never consolidates a working map) — so most tests should confirm plausible levers while a deliberate minority attacks the current understanding, where a surprise is most informative. The loop's product is a causal lever-map of the model — what each mechanism buys, and how mechanisms combine — which is what makes

[1] Found written on Feynman's blackboard at Caltech at his death in February 1988. Never published in a paper or lecture, it survives as a bare aphorism; the second line stood directly beneath the first on the same board. Photographs of the blackboard are widely reproduced.

the operators genuinely *reusable* rather than merely catalogued, and what later tells Loop III which lever the model *lacks*.

Loop II: Fitting. The second loop estimates the parameters, spatial fields and initial conditions of the resulting mechanistic model by fitting it directly to experimental observations. Because every operator admits a differentiable implementation, optimization preserves an explicit correspondence between parameters and biological mechanisms.

Loop III: Discovery. The third loop analyses the residual remaining after optimization. Persistent discrepancies are interpreted as evidence that the current operator composition is incomplete rather than simply poorly parameterized. Turning Loop I's hypothesis-driven validate/falsify cycle outward — from the levers the model *has* to the ones it *lacks* — the agent proposes candidate operators, compares competing mechanistic hypotheses, and extends the operator algebra whenever existing mechanisms fail to explain the observations.

Together, these three loops transform simulation, inverse modelling and hypothesis generation into successive stages of a single scientific workflow. The language itself becomes the object of discovery: new biological knowledge is represented by extending the operator algebra rather than merely adjusting parameter values.

Orchestration: loop agents and a master controller. The three loops are realized as autonomous agents coordinated by a *master*: each performs one kind of work — Understanding, Fitting, or Discovery — and communicates only through typed artifacts on a shared blackboard (the mechanistic model, the success metrics with current values, the residual, and a queue of falsifiable hypotheses). The master never performs the loops' work; it is a state machine whose guards are the metrics — advancing a stage when its metric is met, invoking Discovery when a residual persists, re-invoking Fitting once a new operator is admitted. Beyond scheduling it carries a few responsibilities: it *authors* the success metrics, so "goal achieved" is a falsifiable test rather than a visual impression; it *grounds* hypotheses in the reference literature and code before Discovery proposes; it gates every proposal on *compliance* (a well-typed Plexus contract, not a simulator-specific patch) and *consistency* (an active search for violated invariants — determinism, conservation, agreement between overlapping runs — that separates "the model is incomplete" from "the measurement is wrong"); and it holds each stage open until settled, refusing the "good enough to move on" shortcut.

The master reproduces a modeller's discipline. These responsibilities are not invented for the machine; they formalise how a careful modeller works, distilled from a prototype that deciphers the `tyssue` epithelial vertex model into Plexus operators.[2] Four disciplines recur.

- **Verify the instrument before trusting the measurement** — the sharpest, most-repeated lesson: a "97% hollow" shell that was a recording-stride *misalignment* (vertex positions paired with another frame's connectivity, caught by a determinism contradiction — a long run diverging from its own shorter prefix), and a false "0 invalid faces" that passed self-intersecting bow-tie polygons because the guard checked area > 0, not a simple embedded tiling. An agent that skips this gate elaborates operators to explain an artefact.

[2]Reported separately as a living document of dated field notes: it reconstructs the model as mechanism-named contracts and works each morphogenesis stage by the method below.

- **Small, falsifiable, metric-gated steps — one factor at a time, cheap-first.** Loop I's validate/falsify cadence, enforced across every stage: isolating proliferation rate, division timing and volume stiffness is what homogenised the tissue; a blind sweep would not attribute the win.
- **Perfecting.** A satisfied metric is not qualitative fidelity: the triangular-versus-round-cells gap became a newly authored metric (shape index, sliver fraction) and a target to close.
- **A persistent, revisable master memory** — a log of hypotheses validated and overturned, grounding reads from the reference literature, and the operator atlas — so the search accumulates knowledge instead of re-deriving it each round.

What the loop cannot yet supply — and the agents that must. A prototype run with a human in the loop makes the division of labour precise, and honest about its frontier: the worker agents are strong at execution, breadth and bookkeeping, but four contributions came reliably from the human and name what the scheme must still internalise.

- **Goal custody** — the single most valuable intervention. A worker elaborates whatever it is touching and cannot catch its own drift ("I thought we were starting tubing from 2000 cells"); the master's objective must be held *separately* from the workers'.
- **Temporal perception** — a genuinely structural gap: an agent reads a static strip but cannot *watch a movie*, yet the decisive evidence was in the time course (a pattern that forms only at 12 s; a spot that decays then reforms; tip cells that balloon at 2 s). A dynamics-watching critic is a required agent, not a convenience.
- **Domain priors and taste** — reading the activator gradient off Okuda's figure; naming amplification feedback before finding it is Gierer–Meinhardt.
- **Literature grounding** — "which reaction–diffusion regime does Okuda use?" → the GM unlock.

A faithful multi-agent scheme is therefore not one master and identical workers but a small division of labour — a goal-custody supervisor, a dynamics-watching critic, and a literature-grounding reader — around the disciplined loops above.

A Reference implementation

The main text defines Plexus as a formal language for expressing biological mechanisms. This appendix describes the reference implementation of that language. The implementation follows the abstractions introduced in the paper without introducing additional biological concepts. Its purpose is simply to execute models written in the Plexus language.

The implementation distinguishes two levels. The *language* defines the biological semantics through operators, sets, states and fields. The *runtime* provides concrete software classes implementing these concepts.

Language	Reference implementation
Operator (contract)	`OperatorContract`
Implementation	an `Operator` subclass, registered under a contract
Set	`Level`
State	typed tensors stored by a `Level` (sliced by its `state_schema`)
Map	an index buffer on a `Level` (`parent`, `pre`, `post`)
Field	`Field`
Biological model	`Hierarchy`
Registry	`@register_operator` / `@register_entity` / `@register_field`

Maps are not a separate runtime object. Consistently with the operator-centric view of the main text, a map is an index buffer carried by a `Level` (`parent`/`parent_name` for containment, `pre`/`post` for incidence) and *named* by the operators that traverse it in their typed signature — so the runtime realizes four primitives (operators, sets, states, fields), with maps folded into the operator.

A.1 Hierarchy

The `Hierarchy` represents one executable biological model. It is the runtime container responsible for assembling the language primitives into a simulation.

A Hierarchy stores

- the collection of biological Levels,
- the continuous Fields,
- the instantiated Operators,
- the execution Schedule,
- backend-specific runtime data.

The Hierarchy therefore corresponds to one complete biological model expressed in the Plexus language.

A.2 Levels

A `Level` is the runtime representation of a biological Set. Its elements are stored as flat batched tensors (never one object per element).

Concretely, a Level carries

- a `state` tensor and the `state_schema` that names its columns (`pos`, `vel`, `voltage`, ...);
- an integer `depth` (a scale hint; 0 for a leaf set);
- optional biological `types` (parametric variants within the set);
- its maps, as index buffers: `parent` (with `parent_name`) for containment, and `pre`/`post` for the incidence of an edge-set;
- an occupancy vector `occ`$\in [0, 1]$ marking the live subset, so a cardinality-changing operator (Divide/Die) wakes or retires fixed slots without resizing.

State is stored as contiguous tensors for computational efficiency, while its biological interpretation is determined entirely by the `state_schema` declared by the language. Consequently, the runtime never assumes that a particular state variable represents position, voltage, concentration or any other specific quantity: operators read named blocks (`lvl.get("pos")`) rather than hard-coded column offsets.

A.3 Fields

Fields represent continuous spatial quantities. Unlike Levels, which contain discrete biological entities, Fields describe continuous media such as morphogen concentrations, extracellular signals, pressure or material properties.

Operators interact with Fields exclusively through the maps declared by their typed signatures. The numerical realization of these maps (interpolation, particle–grid transfer, neural implicit evaluation, etc.) is entirely implementation dependent.

A.4 Operators

The runtime realizes the contract/implementation split of Section 5 directly. A biological operator is an `OperatorContract` — a fixed record of `(name, kind, family, set, signature)` together with a dictionary of interchangeable `implementations` and a `default`. Each implementation is an `Operator` subclass; the contract owns the biology, the subclass owns only the numerics.

An operator is registered by decorating its implementation class. The FIRST registration of a name creates the contract from the class's typed `signature()`; a later registration of the same name with a different `implementation=` adds an interchangeable implementation to that same contract (the engine checks the two share a `kind`, so only the numerics may differ).

Listing 1: An operator implementation: the decorator names the contract (`set`/`kind`/`family`) and this implementation; the class attributes are the typed signature and the capabilities.

```
@register_operator("neuron_voltage", set="neuron", kind="lateral",
                   family="interaction", implementation="hodgkin_huxley")
class HodgkinHuxley(Lateral):
    # typed signature -- the biology (Plexus2 sec. 2.1)
    INPUTS  = ["neuron", "synapse"]   # input sets
    OUTPUTS = ["neuron"]              # output sets
    READS   = ["voltage", "w"]        # state consumed
    WRITES  = ["voltage"]             # state produced
    MAPS    = ["pre", "post"]         # maps traversed (folded into the operator)
    # capabilities -- the numerics
    EMIT           = "velocity"       # integration order of the returned delta
    SUPPORTED_DIMS = [2, 3]
    DIFFERENTIABLE = True
    def forward(self, H, mask=None):
        ...                           # returns {"neuron": d_voltage}
```

Three things follow from this shape.

Signature. `signature()` returns the typed morphism {`inputs, outputs, reads, writes, maps`}, defaulting the sets to the acting set `SET` when a class leaves them unset. This is the machine-readable form of "operator as a typed morphism between sets"; the validator and the atlas read it, the engine never does.

Capabilities. Each implementation declares `SUPPORTED_DIMS` and `DIFFERENTIABLE`; the contract's `capabilities()` tabulates them across implementations, so an inverse loop can select, e.g., the differentiable implementation of a contract.

Kind and purity. Every operator belongs to exactly one `KIND` — the eight of the main text (`lateral`, `aggregate`, `broadcast`, `exchange`, `field`, `rewire`, `structural`, `seed`). Dynamics operators are pure: `forward` returns a per-set delta and the engine integrates once per tick (its order set by `EMIT` $\in$ {`velocity`, `acceleration`, `mpm_acceleration`}); `field`/`rewire`/`structural`/`seed` operators mutate the field, the relation, the membership, or the initial state in place and return nothing.

The seed window. `seed` is the one kind whose *schedule* is part of its contract rather than of the specification: the runtime reads the specification once before execution and confines every seed to the opening frames. The window is the largest one any seed in that specification requests, defaulting to a single frame and capped at a small maximum. A model may therefore establish its initial state over the few frames it needs, for instance while a mesh relaxes, but cannot ask an initial condition to run for the length of the trajectory. This is the same move as `EMIT`: a discipline every specification would otherwise have to remember becomes a guarantee the language provides.

Model and implementation are different choices. A contract may be varied along two axes, and conflating them makes a model-search campaign unable to say what it found.

An **implementation** is a numerical realization of a fixed biology. Its variants share the contract's parameters and answer the same question with different numerics: diffusion on a combinatorial graph Laplacian or on interface-weighted finite volumes; a finite-difference or a spectral field update. Swapping one for another holds the operating point fixed, and a result that changes under the swap is a statement about discretization, not about the tissue.

A **model** is a different biological hypothesis occupying the same slot. Gray–Scott, Brusselator and Gierer–Meinhardt are all reaction operators on a two-species state, but they are not three ways of computing one reaction: their parameter sets are disjoint ({`F`,`kk`}, {`A`,`B`}, {`a0`,`mu_a`,`sat`}), so no common operating point exists at which to compare them. Likewise an operator coupling shape back to chemistry may sense curvature, cortical tension, apical area or pressure; these share a signature exactly, yet each is a distinct claim about mechanotransduction, resting on different evidence. Swapping a model *is* an experiment.

Declaring both through one keyword had a concrete cost. In our developmental campaign the search's own documentation described implementation swaps as "genuine mechanism edits" while this appendix described them as numerical choices; every finding recorded against the reaction operator was in fact a finding about Gray–Scott, and the search verb that was pushed hardest by coverage analysis was the one whose meaning was ambiguous. The registry's only guard — that variants of a contract share a `KIND` — cannot detect this, since all four shape-sensing variants are `lateral`.

Contracts therefore carry `model=` and `implementation=` as separate axes: a contract has models, and a model has implementations. A specification names them with separate keys, and naming one where the other is meant is refused rather than silently accepted. Two consequences follow. A mechanism claim becomes attributable, because the record names the model it is about. And a

search can distinguish an edit that changes the biology from one that changes only the arithmetic — which is the difference between an experiment and a control.

Selection is by name: `get_operator(name, model, implementation)` returns the chosen class (the defaults when none are named), and `get_contract(name)` exposes the whole contract. The registry therefore separates biological semantics from numerical realization: new numerical methods are new registered implementations, new hypotheses are new registered models, and neither is an edit to the execution engine.

A.5 Schedules

A biological model is executed by evaluating an ordered Schedule of operator applications.

During each simulation step

1. operators are evaluated in Schedule order,
2. each dynamics operator returns a per-set delta (or a field/relation/structural operator mutates the grid, the edge set, or the membership in place),
3. the engine sums the deltas acting on each set,
4. each set is integrated once, in the order its operators declare (`EMIT`: first-order `velocity` or second-order `acceleration`).

Because operators are pure and nothing is written in place except the explicitly allowed buffers, gradients flow through the sums and the integrator, so the whole rollout stays differentiable. The Schedule therefore implements the operational semantics of the main text while remaining independent of the particular numerical implementations it composes.

A.6 Model specification

Biological models are described declaratively rather than procedurally, as a single `spec.yaml` with blocks `general` / `sets` / `fields` / `operators` / `schedule`. An operator line names the contract (`op`) and, optionally, which implementation to run (`implementation:`); omitting it selects the contract's default, so a single-implementation spec is written exactly as before.

A specification therefore defines

- biological Levels and their state schemas,
- Fields,
- the operators to instantiate, each with its selected implementation,
- the execution Schedule.

Before execution the specification is validated against the registered contracts: an unknown operator, an unknown `implementation` for a known operator, a missing required parameter, a selector on an undeclared set/field, or an unresolved schedule token is rejected up front — so a spec that loads is runnable, and the capability checks (e.g. `SUPPORTED_DIMS`) run against the implementation that will actually execute.

A.7 Reference architecture

The reference implementation follows a strict separation of concerns.

- The language (`sets` / `states` / `fields` / `operators`) defines biological semantics.
- The registry stores each operator as an `OperatorContract` with its implementations.
- Implementations (`Operator` subclasses) realize those contracts numerically.
- The validator (`schema.py`) admits only well-formed, runnable specs.
- The execution engine (`engine.py`) builds the `Hierarchy` and iterates the Schedule; it never contains mechanism-specific code.

A single audit (`tools/audit_operator_registry.py`) enforces the contract across the whole registry — every operator has a valid `family` and `kind`, a canonical `EMIT`, and a `SUPPORTED_DIMS`$\subseteq \{2, 3\}$ — so the vocabulary cannot silently drift. Consequently, extending Plexus never requires modifying the execution engine: new biological mechanisms are new operator contracts, and new numerical methods are additional implementations of existing operators.

The reference implementation therefore mirrors the architecture proposed in the main text: the language remains stable while implementations and numerical backends evolve independently.

B Building the Plexus operator atlas

One of the principal objectives of the Plexus research program is to construct a comprehensive atlas of biological operators by systematically analysing the existing corpus of biological simulation models. Rather than cataloguing simulators, the atlas catalogues reusable biological mechanisms together with their numerical implementations.

This atlas serves two complementary purposes. First, it provides the biological vocabulary from which new models are assembled. Second, it defines the search space explored by the agentic discovery framework described in the main text.

B.1 From simulators to operators

Existing simulation frameworks are not imported directly into Plexus. Instead, each framework is systematically decomposed into the language introduced in this paper.

The decomposition proceeds through the following steps:

1. identify the biological sets,
2. identify the associated state variables,
3. identify continuous fields,
4. identify biological operators,
5. identify the maps required by each operator,
6. separate biological semantics from numerical implementations,
7. register each operator in the Plexus language.

The result is an implementation-independent description expressed entirely in terms of operators, sets, states and fields.

B.2 Normalization

Different simulators frequently implement identical biological mechanisms using very different numerical algorithms. The atlas therefore groups operators according to biological semantics rather than implementation.

For example,

Biological operator	Possible implementations	Example frameworks
Mechanical interaction	MPM, FEM, Vertex, Phase Field	Taichi, Chaste, Tissue Forge
Chemotaxis	Finite differences, Neural fields	Morpheus, CompuCell3D
Electrical propagation	Hodgkin–Huxley, Neural ODE, GNN	NEURON, Jaxley, BrainPy
Diffusion	Finite differences, FEM	Morpheus, VirtualLeaf
Cell division	Vertex, CPM, Agent-based	Chaste, CompuCell3D

The objective is therefore not to catalogue implementations but to identify the smallest reusable vocabulary capable of reconstructing the existing simulation literature.

B.3 Operator metadata

Every entry in the atlas records

- the operator name, family and kind,
- its typed signature (input/output sets, state read/written, maps),
- its implementations and their capabilities (supported dimensions, differentiability),
- its biological interpretation (`MECHANISM_TAGS`) and the role of each tunable parameter (`PARAM_ROLES`),
- originating publications and reference repositories,
- validation status.

The current registry already carries most of this per operator — name, family, kind, acting set, typed signature, implementations, capabilities, mechanism tags and parameter roles, with provenance kept in the operator's docstring; the atlas adds the cross-framework publication and repository record. Consequently the atlas simultaneously documents biological mechanisms, available numerical realizations, and scientific provenance.

B.4 Atlas growth

The atlas is intended to evolve continuously.

Every newly analysed simulator contributes either

1. an implementation of an existing operator,
2. a refinement of an existing typed signature,
3. or a genuinely new biological operator.

The first outcome strengthens numerical diversity. The second improves the language. The third expands the biological vocabulary itself.

B.5 Validation of the operator algebra

The atlas provides a direct experimental test of the central hypothesis of Plexus.

If the decomposition of a sufficiently broad collection of biological simulation frameworks converges toward a compact and reusable operator vocabulary, then the proposed operator algebra constitutes a meaningful intermediate representation for biological modelling.

Conversely, repeated discovery of genuinely new operator families indicates that the language remains incomplete and must be extended.

The operator atlas therefore serves both as a repository of biological knowledge and as the principal mechanism through which the Plexus language itself evolves.

C Implementing new operators

The Plexus language grows by introducing new biological operators rather than by modifying the execution engine. A new operator should be introduced only when an existing operator cannot express the mechanism without changing its biological meaning. New solvers, discretizations, parameterizations or hardware targets should instead be added as alternative implementations of an existing operator.

C.1 Define the biological mechanism

Before implementation, the mechanism must be described independently of its numerical realization. This description should specify

- the biological transformation represented by the operator,
- its operator family,
- its input and output sets or fields,
- the state variables it reads and produces,
- the maps required to relate the participating objects.

Together, these elements define the operator's typed signature, which serves as its biological contract.

C.2 Separate semantics from implementation

The operator contract and its numerical implementations are represented separately.

A single biological operator may admit multiple implementations based, for example, on Material Point Methods, finite elements, vertex models, phase fields, Neural ODEs or graph neural networks. These implementations may differ in their numerical assumptions, spatial representation, supported dimensions or differentiability while preserving the same biological semantics.

Conversely, mechanisms should not be merged merely because they share similar code or numerical utilities. Distinct biological transformations require distinct operator contracts.

C.3 Register the operator and its implementations

Once defined, the operator is added to the registry by decorating its implementation class with `@register_operator(name, set=, kind=, family=, implementation=)` (Listing 1). The decorator creates the contract on the first implementation of a name and records

- the canonical operator name (plus any aliases), family and kind,
- its typed signature, declared as the class attributes `INPUTS` / `OUTPUTS` / `READS` / `WRITES` / `MAPS`,
- the available implementations, keyed by `implementation` name,
- their capabilities (`SUPPORTED_DIMS`, `DIFFERENTIABLE`),
- their scientific provenance (in the class docstring).

A second numerical method for the same mechanism reuses the same `name` with a new `implementation=` and is checked to share the contract's `kind`. Registration makes the mechanism available to every compatible Plexus model without introducing biological knowledge into the execution engine.

C.4 Validation

Every implementation should satisfy four requirements:

1. it satisfies the declared typed signature;
2. it reproduces the intended biological mechanism;
3. its numerical behaviour is verified;
4. its scientific provenance is preserved.

In the reference implementation the first three are gated concretely: the registry audit (`audit_operator_registry.`
checks the declared contract, schema validation checks that a spec using the operator loads, and a minimal live spec plus the test suite check its numerical behaviour.

Validation establishes structural and numerical consistency, but does not by itself establish biological validity. The latter must be assessed through comparison with observations, known limits, ablations or the originating model.

C.5 Promotion

Prototype implementations should not enter the core registry directly. Promotion requires a stable operator contract, at least one validated implementation, clear scientific provenance, and evidence that the mechanism is reusable beyond its originating prototype.

The guiding principle is

> *Prototype freely; promote biological mechanisms conservatively.*

D Refactoring guidelines

The current Plexus repository predates the formalization of the language presented in this paper. Consequently, much of the biological semantics is embedded directly in numerical implementations.

Refactoring aims to recover these semantics and progressively reorganize the codebase around the language abstractions introduced in this work.

D.1 General principles

Every refactoring should follow the same principles.

- Preserve biological behaviour.
- Improve separation between biological semantics and numerical methods.
- Replace implicit assumptions by explicit typed signatures.
- Prefer extending the language rather than adding special cases.
- Refactor incrementally while preserving existing functionality.

D.2 Recommended workflow

When refactoring an existing module, contributors should proceed in the following order.

1. Identify the biological mechanism implemented by the code.
2. Determine whether it corresponds to an existing operator.
3. If not, introduce a new operator contract.
4. Separate the biological contract from the numerical implementation.
5. Register the implementation.
6. Validate that the refactored implementation reproduces the original behaviour.

D.3 Code migration

Existing code should progressively migrate toward the following organization.

- Biological concepts belong to operator contracts.
- Numerical algorithms belong to implementations.
- Execution logic belongs to the runtime.
- Data structures belong to Levels, Fields and Hierarchies.

The execution engine should never contain mechanism-specific code.

D.4 Incremental evolution

Refactoring is expected to be continuous. During the transition, legacy modules and language-native modules may coexist. New developments should preferentially follow the language architecture, while legacy components are progressively decomposed into reusable operators as they are revisited.

The objective is not to rewrite the repository, but to allow the language to emerge progressively from the existing implementation.

E Reference corpus for the operator atlas

The Plexus operator atlas is constructed by systematically decomposing existing simulation frameworks into biological operators, typed signatures and numerical implementations. To ensure reproducibility, the initial atlas is built from open-source projects for which both the scientific publication and an implementation are publicly available.

Rather than treating each simulator as an indivisible software package, every framework is analysed according to the methodology described in Appendix B. The objective is to recover the reusable biological mechanisms implemented by each project and register them as operators within the Plexus language.

The reference corpus is organized into several thematic categories.

E.1 Cell and tissue mechanics

These frameworks describe multicellular mechanics, growth, adhesion, proliferation, remodelling and morphogenesis.

- Chaste
- CompuCell3D
- Morpheus
- Tissue Forge
- VirtualLeaf
- PhysiCell
- Active Vertex Model for Cell-Resolution Description of Epithelial Tissue Mechanics
- Multicellular Simulations with Shape and Volume Constraints using Optimal Transport
- Cell Simulation as Cell Segmentation

Typical operators extracted from these models include

- mechanical interaction,
- adhesion,
- active stress,
- cell growth,
- cell division,
- apoptosis,
- neighbour rewiring,
- particle–cell aggregation,
- optimal transport.

E.2 Vertex models and 3D morphogenesis

Feynman's second blackboard line urges not just creation but the understanding of *everything that has already been built*. Reproducing a *known* result — Okuda *et al.*'s autonomous 3D tubulation and branching, a Turing reaction–diffusion coupled to a 3D vertex model — inside Plexus therefore begins by surveying, as exhaustively as we can, the vertex-model and morphogenesis literature it descends from and the code that ships with it. This focused sub-corpus is that survey.

The Okuda lineage (the reproduction target).

- Combining Turing and 3D vertex models reproduces autonomous multicellular morphogenesis with undulation, tubulation, and branching — Okuda et al. (2018), the target.
- Three-dimensional vertex model for simulating multicellular morphogenesis — Okuda et al. (2015), the base 3D vertex model (volume/surface energy, reversible network reconnection).
- Reversible network reconnection model for simulating large deformation in dynamic tissue morphogenesis — Okuda et al. (2013), the RNR / T1 topology operator.

Vertex-model frameworks that ship code.

- tyssue — the true-vertex (AVM) Python framework; the mechanical substrate of our prototype.
- tvm (Zhang & Schwarz, 2022; code) — a maintained 3D space-filling vertex model, a numerical-validation target.
- Graph vertex model (Sarkar & Krajnc, 2024; code neoVM) — topology as a graph; 3D reconnection decomposed into elementary T1 moves.
- 3D-Vertex-Model — a Mathematica port of Okuda's 2012/2013 reconnection model.
- Tissue Forge (2023) — a general C++/Python vertex-modelling substrate.

Differentiable vertex / morphogenesis models (nearest to the Plexus architecture).

- VertAX (Pasqui et al., 2026; code) — a differentiable JAX vertex model; benchmarks implicit differentiation through equilibrium against an unrolled relaxation loop.
- jax-morph (Deshpande et al., 2025; code) — composable differentiable growth / division / diffusion modules, the closest existing analogue of an operator registry.
- Differentiable Design for Morphogenesis (Beker & Dumitrascu, 2026) — a differentiable forward tissue model with a GNN-based inverse.
- Mean-field elastic moduli of a 3D cell-based vertex model (Kim, Zhang & Schwarz, 2024) — closed-form moduli to validate a 3D implementation against.

Reaction–diffusion on growing domains (the dilution term the patterning must carry).

- Turing patterns on a two-component isotropic growing system, Part 1 (Ledesma-Durán, 2023) — growth adds a dilution term that makes the homogeneous base state time-dependent.
- Base state of growing reaction-dilution systems (Ledesma-Durán, 2026) — how patterning conditions change once reaction rates must balance local volume change.

Tube, lumen and sheet morphogenesis (the target morphology).

- Three-dimensional morphogenesis of epithelial tubes (Yu & Li, 2024).
- Mechanochemical dynamics in multicellular lumens (Yu et al., 2024).
- A virtual spherical-shell multicellular platform (Fujiwara et al., 2022) — the same shell geometry we build on.
- Growth and shrinkage of tissue sheets on substrates: buds, buckles, and pores (Noguchi & Elgeti, 2024).
- Computational approaches for simulating luminogenesis (Fuji et al., 2022).
- Modeling polarity-driven laminar patterns in bilayer tissues (Moore, Dale & Woolley, 2023).

Surveys and further corpus.

- No country for old frameworks? Vertex models and their ongoing reinvention (Briñas-Pascual et al., 2024).
- Cell-based computational models of organoids: a systematic review (Neagu et al., 2026).
- Beads, springs and fields in cell biophysics (Sorichetti et al., 2026).
- Modelling cellular interactions and dynamics during kidney morphogenesis (Cook et al., 2022).
- Avalanches during epithelial tissue growth: a Drosophila eye-disc model (Courcoubetis et al., 2022).

E.3 Reaction–diffusion and signalling

These models describe transport, biochemical signalling and spatial pattern formation.

Representative projects include

- Morpheus
- Reaction-Diffusion Systems in Intracellular Transport and Control
- A Programmable Reaction-Diffusion System for Spatiotemporal Cell Signalling Circuit Design
- SPDEs

Representative operators include

- diffusion,
- reaction,
- secretion,
- uptake,
- field sampling,
- broadcast signalling.

E.4 Neural circuits

These frameworks model neuronal signalling and connectome dynamics.

Representative projects include

- Jaxley
- BrainPy
- Connectome-GNN

Typical operators include

- synaptic transmission,
- membrane integration,
- plasticity,
- neuromodulation.

E.5 Mechanobiology and collective behaviour

Several recent frameworks couple mechanics with biochemical signalling and collective migration.

Representative examples include

- ERK-mediated mechanochemical wave models
- Active Vertex mechanics
- Collective cell migration models
- Bioelectric morphogenesis

Representative operators include

- mechanochemical coupling,
- force transmission,
- wave propagation,
- chemotaxis,
- collective polarity.

E.6 Inverse modelling and differentiable simulation

These projects are particularly relevant for the mechanistic inverse modelling described in Section 6 because they expose differentiable representations of biological systems.

Representative projects include

- MultiCell: Geometric Learning in Multicellular Development
- Differentiable Rendering for 3D Fluorescence Microscopy
- Inferring Cell Junction Tension and Pressure from Cell Geometry

- Machine Learning Interpretable Models of Cell Mechanics from Protein Images
- 3D Single-Cell Shape Analysis using Geometric Deep Learning

Typical operators include

- image formation,
- geometric inference,
- parameter estimation,
- mechanical inversion,
- differentiable observation.

E.7 Image analysis and quantitative biology

These projects provide computational operators that transform experimental observations into structured biological measurements.

Representative projects include

- TissueMiner
- ERNet
- DeXtrusion
- FlowShape
- CellRank

Representative operators include

- segmentation,
- tracking,
- graph construction,
- feature extraction,
- trajectory inference.

E.8 Interactive artificial-life programs

Alongside the research-grade simulation frameworks above, a broad ecosystem of interactive artificial-life and complex-systems programs explores the same emergent mechanisms — self-organization, growth, pattern formation and evolution — often in real time and in the browser. These provide an accessible, complementary corpus for the atlas:

- Lenia
- Evoloops
- Particle Life
- Bibites

- Life Engine
- Convolutional CA
- Game of Life
- Multi-neighborhood CA
- Alien Project
- Germs genetic algorithm
- Subnautica
- Synthetic Selection
- Neural CA
- Particle Lenia

E.9 Building the atlas

Each framework in the reference corpus is analysed using the same procedure.

1. Identify the biological sets.
2. Identify their state variables.
3. Identify continuous fields.
4. Identify the biological operators.
5. Identify the maps relating the participating objects.
6. Separate biological semantics from numerical implementation.
7. Register the resulting operators in the Plexus atlas.

The objective is not to reproduce the original software architecture, but to recover the reusable biological mechanisms that underlie existing simulators. Each analysed framework may contribute additional implementations of existing operators, refine existing typed signatures, or reveal genuinely new biological operators. Consequently, the atlas evolves continuously as new open-source simulation frameworks become available.

F An extended repository survey

Appendix E lists the initial reference corpus. This appendix expands it into a broader candidate survey of fifty open-source repositories, grouped into five mechanism classes. For each repository we record its primary model family and the operator families we expect to extract from it. These operator families are candidates to be confirmed by source-level inspection following the methodology of Appendix B: the entries reflect what each project advertises, not yet a normalized set of registered contracts.

F.1 A. Multicellular and agent-based dynamics

These are likely the highest-yield sources for cell states, cell–cell mechanics, phenotype transitions, growth, division, death, secretion, uptake and agent–field coupling. PhysiCell, for example, targets large 2-D/3-D multicellular systems, while PhysiBoSS adds intracellular Boolean signalling.

#	Repository	Primary model family	Likely operator families
1	MathCancer/PhysiCell	Off-lattice cell agents	divide, die, adhere, repel, migrate, secrete, uptake
2	PhysiBoSS/PhysiBoSS	PhysiCell + Boolean networks	all above + activate, inhibit, switch phenotype
3	MathCancer/BioFVM	Diffusive microenvironment	diffuse, decay, source, sink, Dirichlet boundary
4	Chaste/Chaste	Multiscale cell/tissue simulation	cell cycle, forces, PDE coupling, electrophysiology
5	CompuCell3D/CompuCell3D	Cellular Potts	copy, adhere, constrain volume, chemotax, divide
6	BioDynaMo/biodynamo	General biological agents	grow, divide, move, interact, diffuse signals
7	tissue-forge/tissue-forge	Particle biology and chemistry	force, bond, react, diffuse, secrete, assemble
8	HaseloffLab/CellModeller	Growing microbial cells	elongate, divide, collide, signal, regulate
9	RobertGregg/CellularPotts.jl	Cellular Potts in Julia	adhere, move, divide, constrain area/volume
10	ingewortel/artistoo	Browser Cellular Potts	copy, adhesion, volume constraint, chemotaxis
11	mathbioleiden/Tissue-Simulation-Toolkit	2-D Cellular Potts	adhesion, motility, PDE signalling, growth
12	shabaz/gpu-cpm	GPU Cellular Potts	lattice copy, adhesion energy, volume constraint

CompuCell3D and Artistoo are explicitly Cellular Potts environments; the Tissue Simulation Toolkit is another 2-D Glazier–Graner implementation.

F.2 B. Vertex, Voronoi and epithelial mechanics

These repositories should yield topology-changing and geometric operators that agent-centre models often hide: area and perimeter elasticity, junction tension, T1 transitions, cell extrusion, polygon division and boundary remodelling. CellGPU covers Voronoi and dynamic vertex models, while Tyssue represents epithelial sheets.

#	Repository	Primary model family	Likely operator families
13	sussmanLab/cellGPU	Voronoi/SPV and vertex	area elasticity, perimeter tension, self-propel, T1
14	DamCB/tyssue	Epithelial vertex model	contract, stretch, divide, extrude, reconnect
15	Pablo1990/pyVertexModel	3-D vertex/scutoid	apical/basal tension, lateral adhesion, topology
16	yasuhiroinoue/PyCellVertex	2-D vertex model	growth, division, rearrangement, morphogenesis
17	mmarinriera/VertexModel	Epithelial vertex dynamics	area force, edge tension, neighbor exchange
18	mshagirov/vertex_simulation	Cell-monolayer vertex dynamics	relax vertices, constrain boundary, compute energy
19	kylechanphy/TissueSim	Active vertex model	self-propel, align, deform, rearrange
20	vmansov/clustering_vmodel	Mutant-cell vertex model	cluster, alter mechanics, relax mesh
21	lixinzhi4429/VM_tissue_growth	Growing epithelial vertex model	grow, divide, stress-feedback, align polarity
22	BanerjeeLab/HNP_model	Developmental vertex model	crawl, tension, stress alignment, feedback

The Python vertex implementations explicitly advertise tissue morphogenesis, rearrangements, area/perimeter geometry and boundary constraints.

F.3 C. Spatial molecular and transport processes

This family is essential because it yields operators below the cell scale: Brownian motion, binding, unbinding, unimolecular and bimolecular reactions, membrane crossing, adsorption, desorption, compartment transport and surface diffusion. Smoldyn, ReaDDy and MCell are complementary particle-based spatial simulators; VCell spans deterministic, stochastic, particle and moving-boundary models.

#	Repository	Primary model family	Likely operator families
23	readdy/readdy	Particle reaction–diffusion	diffuse, react, bind, unbind, transform topology
24	ssandrews/Smoldyn	Brownian molecular particles	diffuse, collide, react, adsorb, cross membrane
25	mcellteam/mcell	Monte Carlo cellular microphysiology	diffuse, surface-react, release, absorb
26	mcellteam/cellblender	MCell geometry/-model authoring	define compartments, membranes, sources and reactions
27	CNS-OIST/STEPS	Stochastic reaction–diffusion	react, diffuse, channel flux, membrane current
28	virtualcell/vcell	Spatial systems biology	ODE, reaction–diffusion, advection, electrophysiology
29	GollyGang/ready	Cellular automata/PDE exploration	local update, diffuse, react, mesh evolution
30	spatial-model-editor/spatial-model-editor	Spatial SBML models	compartmentalize, react, diffuse, impose geometry

F.4 D. Intracellular signalling and biochemical networks

These projects can supply operators for reaction networks, rule-based molecular interactions, state transitions, stochastic events, conservation laws and parameterized kinetic laws. VCell and COPASI are foundational mechanistic cell-biology platforms, while BioNetGen describes rule-based interactions without enumerating every molecular species.

#	Repository	Primary model family	Likely operator families
31	copasi/COPASI	Biochemical reaction networks	react, catalyze, inhibit, transport, event
32	RuleWorld/bionetgen	Rule-based biochemical models	bind, unbind, modify, synthesize, degrade
33	sys-bio/tellurium	Reproducible systems biology	integrate ODE, simulate events, compose models
34	sys-bio/roadrunner	SBML simulation engine	integrate kinetics, steady state, sensitivity
35	pysb/pysb	Programmatic rule-based modelling	bind, catalyze, translocate, express, degrade
36	GillesPy2/GillesPy2	Stochastic biochemical kinetics	stochastic reaction, propensity, event
37	StochSS/StochSS	Stochastic systems biology	simulate ensembles, spatial reactions, parameter sweep
38	AMICI-dev/AMICI	Differentiable ODE models	integrate, differentiate, infer parameters
39	opencobra/cobrapy	Constraint-based metabolism	flux balance, uptake, secrete, constrain reaction
40	BhallaLab/moose-core	Multiscale cellular simulation	kinetics, diffusion, electrophysiology, signalling

F.5 E. Neural computation as cellular simulation

Although these are neuroscience frameworks, they are highly relevant to Plexus because they expose another mature corpus of reusable cell-level operators: membrane integration, ion-channel gating, axial conduction, synaptic transmission, spike generation, delays and plasticity. Chaste itself also combines electrophysiology with tissue and cell modelling.

#	Repository	Primary model family	Likely operator families
41	neuronsimulator/nrn	Multicompartment neurons	integrate voltage, gate channels, axial current, spike
42	brian-team/brian2	Equation-defined neural networks	integrate state, threshold, reset, synapse
43	nest/nest-simulator	Large spiking networks	spike, transmit, delay, plasticity
44	arbor-sim/arbor	HPC multicompartment neurons	cable solve, channel current, synaptic event
45	AllenInstitute/bmtk	Brain network models	construct/connect networks, drive, record
46	Neurosim-lab/netpyne	NEURON network specification	instantiate cells, connect, stimulate, simulate
47	LFPy/LFPy	Biophysical neurons and fields	membrane current, extracellular field, electrode sample
48	jonescompneurolab/hnn-core	Cortical circuit biophysics	synaptic drive, dendritic integration, field generation
49	neurolib-dev/neurolib	Neural mass/whole-brain dynamics	local dynamics, network coupling, delayed propagation
50	NeuralEnsemble/Nengo	Neural dynamical systems	encode, transform, filter, learn

F.6 A preliminary atlas record

For each repository the atlas stores a single record separating the raw mechanisms named by the project from their normalized Plexus contracts, together with the evidence and validation status:

Listing 2: One atlas record per repository: raw mechanisms are separated from their normalized Plexus contracts, with evidence and validation status.

```
repository: MathCancer/PhysiCell
paper: DOI-or-PMID
model_family: off_lattice_multicellular
scale:
  - cell
  - tissue
sets:
  - cell
  - substrate_voxel
fields:
  - oxygen
  - signalling_substrate
maps:
  - cell_cell_neighborhood
  - cell_voxel_sampling
operators_raw:
  - cell_cell_force
  - phenotype_update
  - cell_division
```

```
  - apoptosis
  - substrate_secretion
  - substrate_uptake
operators_normalized:
  - repel
  - adhere
  - update_phenotype
  - divide
  - die
  - produce
  - consume
evidence:
  paper_section: ...
  code_path: ...
status: candidate | inspected | normalized | implemented
```

F.7 What this corpus should reveal

The fifty repositories are not fifty independent vocabularies. They repeatedly implement the same mechanisms under different names and numerical schemes. A preliminary normalization should therefore distinguish the biological mechanism from its numerical implementation while preserving a single operator contract. For instance, cell division appears as

- **Mechanism:** `divide`.
- **Implementations:** vertex split, CPM mitosis, off-lattice daughter insertion.
- **Contract:** one cell state and geometry $\rightarrow$ two valid daughter-cell states and an updated neighbourhood.

Likewise, `diffuse` should remain one operator contract even when implemented by finite differences, finite volumes, stochastic particles or a neural implicit field.

Source-level extraction would begin with twelve high-yield representatives — PhysiCell, CompuCell3D, Chaste, Tissue Forge, BioDynaMo, CellGPU, Tyssue, Artistoo, ReaDDy, Smoldyn, VCell and NEURON. Together they span most anticipated Plexus relation classes: set $\rightarrow$ set, set $\rightarrow$ field, field $\rightarrow$ set, field $\rightarrow$ field, topology change, containment and cross-scale coupling. The list is broad enough to begin measuring operator *saturation*: after each repository we record how many genuinely new normalized contracts appear versus aliases of contracts already registered. Saturation — new frameworks contributing implementations rather than new contracts — is the empirical signal that the operator algebra of the main text is complete.